

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

Inserção de Mineração de Dados em Otimização Combinatória: O caso do Problema da Mochila

Thainá Simões Pires

Prof. Orientador:
Pedro Henrique González Silva

**Rio de Janeiro,
Junho de 2021**

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

Inserção de Mineração de Dados em Otimização Combinatória: O caso do Problema da Mochila

Thainá Simões Pires

Projeto final apresentado em cumprimento às
normas do Departamento de Educação
Superior do Centro Federal de Educação
Tecnológica Celso Suckow da Fonseca,
CEFET/RJ, como parte dos requisitos para
obtenção do título de Bacharel em Ciência da
Computação.

Prof. Orientador:
Pedro Henrique González Silva

**Rio de Janeiro,
Junho de 2021**

DEDICATÓRIA

Dedico este trabalho a minha família. Em especial, a minha mãe, que sempre me incentivou e me deu o apoio necessário para que eu chegasse até aqui, me inspirando a cada dia a lutar para que meus sonhos sejam realizados.

AGRADECIMENTOS

Agradeço ao meu orientador, Pedro Gonzalez, que me orientou tanto na iniciação científica quanto neste trabalho, por todo o suporte dado durante esse processo.

Agradeço a minha família e aos meus amigos, que me incentivaram e me apoiaram durante a faculdade, tanto nos momentos felizes quanto nos momentos difíceis.

Por fim, à todos que direta ou indiretamente fizeram parte da minha formação, o meu muito obrigada.

RESUMO

As metaheurísticas são métodos de resolução de problemas, que fornecem estratégias para escapar dos ótimos locais e fazer buscas robustas no espaço de busca [Glover and Kochenberger, 2006]. Elas são muito utilizadas quando se trata de problemas de otimização combinatória onde o uso de métodos exatos é impraticável. O Greedy Randomized Adaptive Search Procedure (GRASP) [Feo et al., 1994] é uma metaheurística muito conhecida e utilizada na literatura, funcionando de forma iterativa, através de uma fase de construção e uma busca local. Uma versão híbrida, chamada DM-GRASP [Ribeiro et al., 2004] [Ribeiro et al., 2006], incorpora técnicas de Mineração de Dados (MD) ao GRASP e vem obtendo resultados promissores. O DM-GRASP consiste na execução de um determinado número de iterações do GRASP, onde são extraídos padrões de soluções de boa qualidade, que são utilizados para guiar a busca por soluções melhores nas iterações que seguem, levando a uma exploração mais efetiva do espaço de busca. O presente trabalho investiga a eficiência da incorporação de técnicas de mineração de dados à metaheurística GRASP, através da aplicação do algoritmo DM-GRASP em um clássico problema de otimização combinatória chamado Problema da Mochila, que possui grande importância dentro do mundo dos problemas de otimização combinatória [Goldbarg and Luna, 2005]. A qualidade dos padrões frequentes será analisada através da comparação dos itens escolhidos para comporem os padrões com a solução ótima, que será gerada para cada uma das instâncias do Problema da Mochila. Experimentos computacionais são realizados para avaliar o impacto dessa abordagem.

Palavras-chaves: metaheurísticas; método híbrido; GRASP; mineração de dados; otimização; pesquisa operacional; problema da mochila;

ABSTRACT

Metaheuristics are solution methods that provide strategies to escape local optimums and do robust searches in the search space [Glover and Kochenberger, 2006]. They are widely used when it comes to combinatorial optimization problems where the use of exact methods is impractical. Greedy Randomized Adaptive Search Procedure (GRASP) [Feo et al., 1994] is a well-known metaheuristic and it's widely used, working iteratively, through a construction phase and a local search. Its hybrid version, called DM-GRASP [Ribeiro et al., 2004] [Ribeiro et al., 2006], incorporates Data Mining Techniques to GRASP and has been getting promising results. DM-GRASP consists of executing a certain number of iterations of GRASP, where patterns of good quality solutions are extracted, which are used to guide the search for better solutions in the iterations that follow, leading to a more effective exploration of the space of search. This work investigates the efficiency of incorporating data mining techniques into the GRASP metaheuristic, through the application of the DM-GRASP algorithm in a classic combinatorial optimization problem called the Knapsack Problem, which has great importance in the world of combinatorial optimization problems [?]. The quality of frequent patterns will be analyzed by comparing the items chosen to compose the patterns with the optimal solution, which will be generated for each instance of the Knapsack Problem. Computational experiments are performed to evaluate the impacts of this approach.

Keywords: knapsack problem; metaheuristics; hybrid method; GRASP; data mining; optimization; operational research

Sumário

1	Introdução	1
2	Fundamentação teórica	4
2.1	Otimização Combinatória	4
2.1.1	Modelagem Matemática	4
2.1.1.1	Programação Linear Inteira	5
2.1.2	Problemas de Otimização Combinatória	6
2.2	Estratégias de Resolução de Problemas	6
2.2.1	Métodos Exatos	7
2.2.2	Métodos Heurísticos	7
2.2.3	Métodos Híbridos	8
2.3	Problema da Mochila	8
2.4	Mineração de Padrões Frequentes	9
2.5	DM-GRASP e Trabalhos Relacionados	10
3	Metodologia	13
3.1	GRASP aplicado ao Problema da Mochila	13
3.1.1	Heurística Construtiva	13
3.1.2	Busca Local	15
3.1.3	GRASP (Algoritmo completo)	16
3.2	Data Mining GRASP (DM-GRASP)	16
4	Avaliação Experimental	19
4.1	GRASP vs DM-GRASP	19
4.2	Análise do comportamento do DM-GRASP	22
4.3	Análise da qualidade dos padrões frequentes	23
5	Conclusões e Trabalhos Futuros	28
	Referências Bibliográficas	28

Lista de Figuras

FIGURA 1:	Etapas da Modelagem - Adaptada de [Goldbarg, 2005]	5
FIGURA 2:	Representação gráfica dos máximos/mínimos globais e locais de f - Adaptada de [Weise, 2007]	7
FIGURA 3:	Exemplo de movimento da busca local	15
FIGURA 4:	GRASP vs DM-GRASP	22

Lista de Tabelas

TABELA 1:	Trabalhos que utilizam o Data Mining GRASP (DM-GRASP) e variações	12
TABELA 2:	Valores das soluções obtidos para as instâncias do Problema da Mochila	20
TABELA 3:	Valores das soluções obtidos para as instâncias do Problema da Mochila	21
TABELA 4:	Porcentagens obtidas para cada uma das instâncias do Problema da Mochila	24
TABELA 5:	Porcentagens obtidas para cada uma das instâncias do Problema da Mochila	25
TABELA 6:	Porcentagens obtidas para cada uma das instâncias do Problema da Mochila	26
TABELA 7:	Porcentagens obtidas para cada uma das instâncias do Problema da Mochila	27

LISTA DE ABREVIACES

DM-GRASP	Data Mining GRASP	ix, 2, 3, 10, 11, 12, 13, 28
GRASP	Greedy Randomized Adaptive Search Procedure	v, 2, 10, 11, 13, 14, 28
LRC	Lista Restrita De Candidatos	13, 14
MD	Minerao De Dados	v, 2, 16
MPF	Minerao De Padres Frequentes	4, 9, 10, 18
PI	Programao Inteira	5
PL	Programao Linear	5
PM	Problema Da Mochila	1, 9

Capítulo 1

Introdução

Problemas de otimização combinatória possuem aplicações em diversas áreas do conhecimento. São problemas de natureza discreta que envolvem a busca pela melhor solução dentro de um conjunto de soluções possíveis. Porém, para muitos destes problemas, torna-se inviável enumerar todas as combinações para achar a melhor solução. Isso se deve ao crescimento exponencial do número de soluções possíveis, que variam com o tamanho do problema [Feo and Resende, 1995].

Como a busca pela solução de problemas como esses é, em muitos casos, uma tarefa computacionalmente difícil, são frequentemente classificados como pertencentes à classe de problemas NP-difícil [Aarts et al., 2003]. Dessa maneira, procurar por novas abordagens que resolvam estes problemas de forma mais eficiente, se torna uma tarefa necessária.

Um clássico exemplo de problema de otimização combinatória que pertence a classe NP-difícil é o Problema da Mochila (PM), que vem sendo estudado em diversos trabalhos e ilustra a situação onde é preciso escolher entre os itens que serão mais valiosos para alocar em uma mochila, sem sobrecarregá-la. Considerando n itens, cada um tem um peso e um valor associado. O objetivo é selecionar um subconjunto de itens, de forma a maximizar o valor da mochila e respeitar a restrição da capacidade [Assi and Haraty, 2019].

O PM é um dos problemas mais estudados e surgiu na literatura possivelmente em [Dantzig, 1957]. Possui diversas variações e pode ser aplicado em áreas como investimentos, logística, escalonamento de processos, financiamento de projetos, entre outras [Assi and Haraty, 2019]. Além disso, possui estreito relacionamento com muitos outros problemas de otimização, evidenciando a sua importância e interesse prático [Goldbarg and Luna, 2005].

Muitos trabalhos foram desenvolvidos a procura de métodos onde não seja necessário listar todas as combinações possíveis [Feo and Resende, 1995] para problemas como o Problema da Mochila. Uma das alternativas são as heurísticas, que encontram soluções aproximadas. O problema dessa alternativa é que elas correm o risco de ficarem presas em ótimos locais que estão bem distantes do ótimo global, podendo portanto não ter efetividade e não encontrar uma boa solução [Goldbarg and Luna, 2005]. Levando isso em consideração, surgem as me-

taheurísticas, procedimentos comumente utilizados para encontrar soluções aproximadas [Barbalho et al., 2013], normalmente em um tempo computacional viável, utilizando-se de um procedimento a mais, que irá guiar a heurística de forma a escapar de ótimos locais.

Dentre as diversas metaheurísticas existentes, o *Greedy Randomized Adaptive Search Procedure* (GRASP) foi escolhido para ser utilizado neste trabalho. Apesar de outras metaheurísticas superarem o GRASP, ele é simples e geralmente rápido, sendo possível produzir soluções consideravelmente boas em um tempo computacional bem curto [Blum and Roli, 2008].

Considerando o objetivo de melhorar metaheurísticas conhecidas, outra alternativa existente é a utilização de métodos híbridos [Talbi, 2002]. De forma mais específica, metaheurísticas híbridas combinam metaheurísticas com outras metaheurísticas [Mauri et al., 2018], formulações matemáticas [González et al., 2019] ou até mesmo com técnicas de mineração de Dados [González et al., 2018].

A Mineração de Dados, processo de extração de padrões que representam conhecimento implicitamente presentes em grandes bases de dados, surgiu durante o final dos anos 80 e evoluiu ainda mais durante os anos 90 [Han et al., 2011]. Hoje em dia, continua tendo cada vez mais destaque.

Tratando-se da quantidade de dados existentes no mundo hoje, em conjunto com a necessidade de técnicas que permitam analisar estes dados, é evidenciada a importância da Mineração de Dados. É possível perceber que o volume de dados é tanto, que já excedeu a capacidade de compreensão humana [Han et al., 2011]. A MD poderá influenciar tanto na resolução de problemas [Witten et al., 2017] quanto no processo de tomada de decisão [Plastino et al., 2014].

Existe na literatura uma versão híbrida da metaheurística GRASP, que incorpora a Mineração de Dados e que surgiu pela primeira vez em [Feo et al., 1994]. Esta versão alcançou resultados importantes para diversos problemas de otimização ao longo dos anos. O algoritmo, chamado DM-GRASP, consiste na execução de um determinado número de iterações do GRASP, onde são extraídos padrões de soluções de boa qualidade. Após a extração, os padrões são utilizados para construir as soluções nas iterações seguintes de forma mais eficiente. Neste trabalho, os padrões a serem extraídos são os conjuntos de elementos que aparecem com frequência em soluções. O intuito é utilizar esses padrões de forma a guiar a busca por soluções melhores nas iterações que seguem, levando a uma exploração mais efetiva do espaço de busca [Plastino et al., 2014].

O principal foco deste trabalho será de investigar a combinação da metaheurística GRASP

com a Mineração de Dados, analisando a qualidade dos padrões frequentes através da comparação dos itens escolhidos para compor os padrões com a solução ótima de cada uma das instâncias. O algoritmo será aplicado ao Problema da Mochila, utilizando instâncias de variados tamanhos. Através de experimentos computacionais, a influência dos padrões frequentes na construção das soluções será avaliada. Para realizar esta avaliação, um algoritmo exato será aplicado nas instâncias escolhidas, de forma a servir como base para o entendimento da qualidade dos padrões frequentes com o DM-GRASP. Espera-se que os padrões frequentes encontrados contribuam para achar soluções de qualidade e de que façam parte da solução ótima.

Além desta introdução, o trabalho está organizado da seguinte forma: O Capítulo 2 aborda os principais conceitos necessários para a fundamentação do trabalho. No Capítulo 3, a metodologia é apresentada. No Capítulo 4 são apresentados os resultados. Por fim, o Capítulo 5 apresenta a conclusão do trabalho.

Capítulo 2

Fundamentação teórica

Este capítulo é dividido em 5 seções. Na Seção 2.1 são introduzidos os tópicos de otimização combinatória, modelagem matemática e problemas de otimização. Na Seção 2.2 são explicadas as diferentes estratégias para resolução de problemas de otimização combinatória. Na Seção 2.3 é apresentada a definição e a formulação do Problema da Mochila. Na Seção 2.4 são apresentados os conceitos relacionados a Mineração de Padrões Frequentes (MPF), a técnica de mineração de dados que será utilizada. Por fim, o DM-GRASP e os trabalhos relacionados são apresentados na Seção 2.5.

2.1 Otimização Combinatória

A otimização combinatória é uma área que faz parte da ciência da computação, da matemática e da pesquisa operacional [Aarts et al., 2003] e que estuda os mais diversos problemas de otimização combinatória. Estes problemas consistem em achar a melhor solução dentro de um conjunto de soluções, que satisfazem um determinado objetivo. Para facilitar o entendimento do leitor, primeiramente serão apresentados alguns conceitos relacionados a modelagem matemática.

2.1.1 Modelagem Matemática

De modo geral, a modelagem de um problema é caracterizada como o processo de busca de uma visão estruturada para um fenômeno da realidade [Goldbarg, 2005]. O passo a passo geral do processo de modelagem pode ser visualizado na Figura 1.

Dentre as fases da modelagem de determinado problema, destaca-se a fase de definição e a de formulação. Na fase de definição, três características principais são definidas, sendo elas: (1) Objetivos (2) Variáveis de decisão e (3) Níveis de detalhe. Além disso, a fase de formulação também é crucial e tem uma carga matemática e quantitativa muito forte [Goldbarg, 2005].

No que tange à Formulação Matemática, são utilizados os Modelos de Programação Ma-

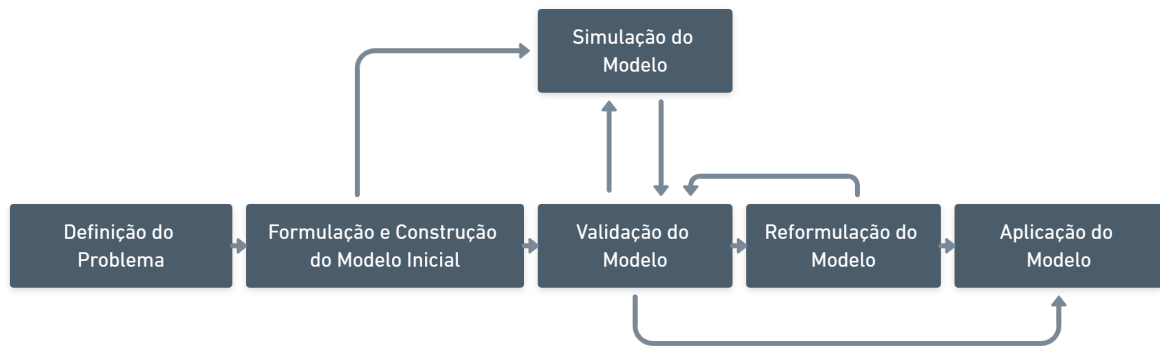


Figura 1: Etapas da Modelagem - Adaptada de [Goldbarg, 2005]

temática. Neste contexto, a palavra programação é utilizada no sentido de planejamento. É neste quesito que a Programação Linear desempenha um papel muito importante na teoria da otimização [Papadimitriou and Steiglitz, 1982].

2.1.1.1 Programação Linear Inteira

A Programação Linear (PL) consiste em uma subárea da Modelagem Matemática. O objetivo deste paradigma é de determinar os valores das variáveis de decisão que maximizam ou minimizam uma função objetivo de característica linear, de forma que as variáveis de decisão estarão sujeitas a restrições também lineares [Chong and Zak, 2013].

A Programação Inteira (PI), também chamada de Programação Linear Inteira, é um tipo específico de Programação Linear, onde um subconjunto de um conjunto de variáveis é composto por variáveis inteiras [Goldbarg and Luna, 2005]. Isto porque ao pensar em termos práticos, algumas das variáveis não terão sentido ao assumirem valores quebrados, como por exemplo em uma situação que é necessário alocar pessoas, máquinas e veículos para atividades [Hillier and Lieberman, 2013]

Quando todas as variáveis assumem valores inteiros, o problema é classificado como de Programação Inteira Pura, enquanto que quando somente algumas das variáveis possuem a restrição de serem inteiras, é classificado como de Programação Inteira Mista. Além disso, caso todas as variáveis sejam binárias, o problema é classificado como de Programação Inteira Binária [Hillier and Lieberman, 2013].

2.1.2 Problemas de Otimização Combinatória

Segundo Blum and Roli [2008], um problema de otimização P pode ser definido como uma tripla (S, Ω, f) , onde S corresponde ao espaço de busca (ou domínio) do problema, definido por um conjunto finito de variáveis de decisão X_i , $i = 1, \dots, n$. O Ω corresponde ao conjunto de restrições relacionadas as variáveis. Caso as variáveis tenham domínio discreto, caracteriza-se um problema de otimização combinatória. No caso das variáveis serem contínuas, o problema é de otimização contínua. O foco deste trabalho está nos problemas de otimização combinatória.

Considere f uma função com domínio S , chamada de função objetivo. Esta função, especifica um valor de custo positivo para cada elemento (ou solução) de S . O objetivo é achar uma solução $s \in S$ tal que:

$$f(s) \leq f(s') \forall s' \in S \text{ quando é um problema de minimização} \quad (2.1)$$

$$f(s) \geq f(s') \forall s' \in S \text{ quando é um problema de maximização} \quad (2.2)$$

No que tange à função objetivo, seus extremos podem ser classificados como global ou local. Dizer que um ponto é o ótimo global significa dizer que é o maior (ou menor) valor da função objetivo no espaço de busca como um todo. No caso de um ótimo local, será o maior (ou menor) valor da função objetivo em uma determinada vizinhança, isto é, em um subespaço de busca [Weise, 2007]. Quando lidamos com um problema de minimização, desejamos encontrar o mínimo global, enquanto que nos problemas de maximização desejamos encontrar o máximo global. Isto é ilustrado através do exemplo da Figura 2, que apresenta um espaço de busca bidimensional de uma função f , com seus respectivos máximos e mínimos globais e locais.

2.2 Estratégias de Resolução de Problemas

Muitos problemas de otimização combinatória pertencem a classe de problemas NP-difícil, isto é, não são resolvidos otimamente em tempo computacional polinomial [Aarts et al., 2003]. Nestes casos, simplesmente enumerar todas as soluções possíveis e retornar a de melhor valor, se torna inviável. Devido a este desafio, surgem na literatura diversos métodos onde nem sempre o ótimo global será encontrado, mas que poderão obter "boas" soluções em um tempo computacional viável [Blum and Roli, 2003].

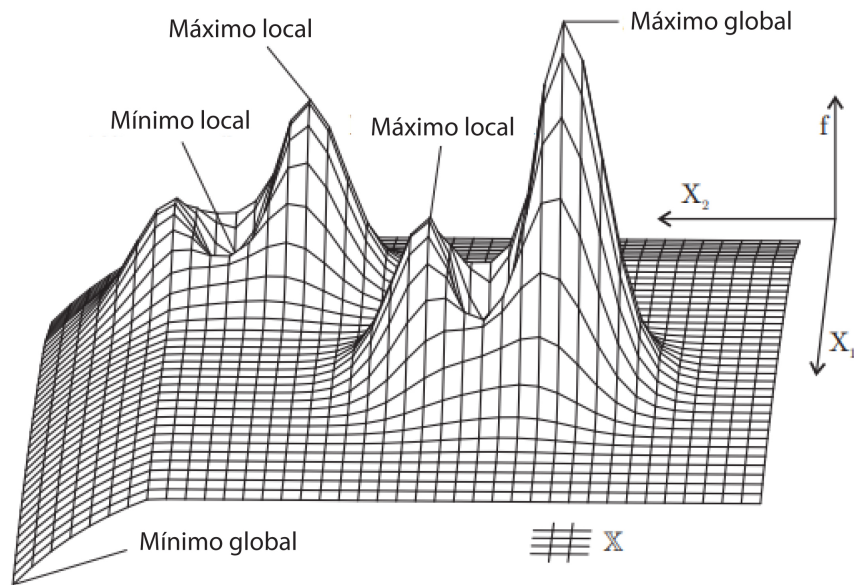


Figura 2: Representação gráfica dos máximos/mínimos globais e locais de f - Adaptada de [Weise, 2007]

2.2.1 Métodos Exatos

Métodos exatos são aqueles que garantem encontrar a solução ótima, sendo possível provar a otimalidade para cada instância de um problema de otimização combinatória. O problema está no fato de que o tempo de execução poderá aumentar de maneira drástica, conforme o tamanho da instância aumentar. Frequentemente, somente instâncias de tamanho pequeno ou moderado poderão ser solucionadas com estes tipos de métodos [Puchinger and Raidl, 2005]. Existem diversas abordagens clássicas de métodos exatos, como o Simplex para a Programação Linear, o Branch-and-Bound [Land and Doig, 1960] para a Programação Inteira, a Programação Dinâmica [Bellman, 1957], entre outros.

2.2.2 Métodos Heurísticos

Em Goldberg and Luna [2005] uma heurística é definida como "Uma técnica que busca alcançar uma boa solução utilizando um esforço computacional considerado razoável, sendo capaz de garantir a viabilidade ou a otimalidade da solução encontrada ou, ainda, em muitos casos, ambas, especialmente nas ocasiões em que essa busca partir de uma solução viável próxima ao ótimo".

A grande desvantagem das heurísticas está na dificuldade de escapar de ótimos locais, o que

deu origem a uma alternativa, chamada de metaheurística. O termo foi apresentado pela primeira vez em [Glover, 1986]. Uma metaheurística é um método iterativo que utiliza estratégias de alto nível, que guiam heurísticas através da combinação inteligente de diversos conceitos para explorar o espaço de busca. Portanto, o objetivo é realizar a exploração de forma eficiente, visando encontrar soluções próximas do ponto ótimo [Osman and Laporte, 1996].

Além da metaheurística Greedy Randomized Adaptative Procedure (GRASP) [Feo et al., 1994], que é utilizada neste trabalho, é possível destacar outras metaheurísticas como: Algoritmos Genéticos [Goldenberg, 1989], Simulated Annealing [Kirkpatrick et al., 1983], Busca Tabu [Glover, 1986], Colônia de Formigas [Taillard, 2002], Variable Neighborhood Search [Mladenović and Hansen, 1997], entre outras.

2.2.3 Métodos Híbridos

Os métodos híbridos consistem na combinação de métodos de diferentes classes. De forma mais específica, as metaheurísticas híbridas combinam uma metaheurística com outra metaheurística ou outras técnicas, de forma a fornecer um comportamento mais eficiente e uma flexibilização maior ao lidar com problemas do mundo real e de larga escala [Blum and Roli, 2008].

As metaheurísticas híbridas são classificadas por [Blum and Roli, 2008] em duas categorias principais. A primeira consiste no desenvolvimento de um *solver*, que inclui componentes de uma metaheurística em outra, enquanto que a segunda combina metaheurísticas com técnicas típicas de outras áreas, como Pesquisa Operacional, Inteligência Artificial e Mineração de Dados.

Além disso, outra propriedade que diferencia abordagens híbridas é a ordem de execução. Em um modelo *batch*, um algoritmo é executado depois de outro, e a informação é passada em apenas uma direção. Em contrapartida, existem os modelos paralelos, onde os algoritmos podem interagir de maneiras mais sofisticadas [Raidl, 2006].

2.3 Problema da Mochila

Dentre as diversas variações que o problema pode ter, a que foi escolhida para ser utilizada neste trabalho é chamada de Problema da Mochila 0-1, muitas vezes definida como Problema da Mochila Unidimensional, pois possui apenas uma restrição [Goldbarg and Luna, 2005].

O Problema da Mochila consiste em, dado um conjunto de n itens, onde cada item $i \in I$ tem

um valor (v_i) e um peso (w_i) associado a ele, o desafio é de seleccionar um subconjunto de itens $\mathcal{M} \subseteq I$ para serem incluídos em uma mochila de capacidade W , sem excederem a capacidade da mesma, maximizando o valor total dos itens adicionados na mochila. Matematicamente, o PM pode ser definido como:

$$PM = \{ \mathcal{M} \subseteq I \mid \sum_{i \in \mathcal{M}} w_i \leq W \wedge \sum_{i \in \mathcal{M}} v_i \geq \sum_{i \in \tilde{\mathcal{M}}} v_i, \forall \tilde{\mathcal{M}} \subseteq I \mid \tilde{\mathcal{M}} \neq \mathcal{M} \wedge \sum_{i \in \tilde{\mathcal{M}}} w_i \leq W \} \quad (2.3)$$

Através da Programação Linear Inteira, é possível formular o problema da mochila da seguinte maneira:

$$\begin{aligned} & \text{maximizar} && \sum_{i=1}^n v_i s_i \\ & \text{sujeito a} && \sum_{i=1}^n w_i s_i \leq W \\ & && s_i \in \{0, 1\} \forall i = 1, \dots, n \end{aligned}$$

Se um item for alocado na mochila, a variável de decisão s_i assume 1 (um) como valor, caso contrário, assume o valor 0 (zero).

2.4 Mineração de Padrões Frequentes

A Mineração de Padrões Frequentes é um dos tópicos de pesquisa mais importantes na área de Mineração de Dados [Liu, 2012]. É possível definir a Mineração de Dados como o processo de descoberta de padrões e regras presentes em bases de dados [Hand and Adams, 2014; Witten et al., 2017].

Um dos conceitos de grande importância dentro da mineração de dados são as regras de associação, que descrevem a frequência com que determinados itens aparecem juntos em um determinado domínio [Han et al., 2011]. Seguindo as definições presentes em [Santos et al., 2005; Silva et al., 2020], o processo de MPF será formalmente descrito. Dado um conjunto de itens $I = \{i_1, i_2, \dots, i_n\}$ e os subconjuntos $X \subseteq I$ e $Y \subseteq I$, onde X e Y são conjuntos não vazios, algoritmos de mineração de dados produzem uma regra de associação $X \Rightarrow Y$, que descreve um determinado padrão. D é definido como um conjunto de transações definidas sobre I , onde cada transação t é um subconjunto de I ($t \subseteq I$). A regra de associação é encontrada na base de dados D com suporte s e confiança c se, respectivamente, $s\%$ das transações em D conterem X

$\cup Y$, e $c\%$ das transações em D que contiverem X também conterem Y . Um exemplo de regra de associação é apresentado a seguir:

$$\text{câmera} \Rightarrow \text{cartão de memória (suporte} = 2\%, \text{ confiança} = 60\%).$$

Isto significa que 60% dos clientes que compraram uma câmera, também compraram um cartão de memória. Um suporte de 2% significa que 2% de todas as transações que estão sobre análise mostram que uma câmera e um cartão de memória são comprados juntos. Os valores do suporte e da confiança são especificados pelo usuário, sendo necessário a execução de experimentos computacionais para determinar os valores [Silva et al., 2020]

O processo de extrair regras de associação de bases de dados transacionais é frequentemente dividido em duas fases. Os termos sup_{min} e sup_{conf} representam, respectivamente, o suporte mínimo e a confiança mínima. A primeira fase, que é chamada de Mineração de padrões frequentes, consiste em identificar todos os conjuntos de itens frequentes, ou seja, conjuntos de itens que ocorrem em pelo menos $sup_{min}\%$ das transações da base de dados. A segunda fase resulta, para cada conjunto de itens frequentes Z , todas as regras de associação $A \Rightarrow B$ com confiança maior ou igual a sup_{conf} , de forma que $A \subset Z$, $B \subset Z$, e $A \cup B = Z$. A fase de Mineração de Padrões Frequentes (MPF) demanda mais poder computacional do que a segunda, sendo muito mencionada na literatura [Goethals, 2003].

2.5 DM-GRASP e Trabalhos Relacionados

A Hibridização da metaheurística GRASP com técnicas de MD, que será utilizada neste trabalho, foi proposta pela primeira vez em [Ribeiro et al., 2004] [Ribeiro et al., 2006], onde a heurística híbrida resultante é chamada de Data-Mining GRASP (DM-GRASP) [Santos et al., 2008]. A ideia principal era de que após um determinado número de iterações do GRASP, o processo de mineração de dados extrairia padrões de um conjunto elite de soluções, que poderiam ser utilizados para guiar as iterações seguintes, de forma a fazer uma busca mais efetiva no espaço de soluções [Maia et al., 2015]. Para validar esta hipótese, os autores aplicaram a ideia para o problema do empacotamento de conjuntos (do inglês, *Set Packing Problem*). Os resultados obtidos mostraram que a utilização dos padrões fez com que o GRASP não só obtivesse soluções de melhor qualidade, mas também as produzisse em melhores tempos computacionais.

O DM-GRASP já foi aplicado com sucesso em outros problemas de otimização combinatória, como o problema de diversidade máxima [Santos et al., 2005], o problema da replicação

de servidores para transmissão multicast confiável [Santos et al., 2006] [Plastino et al., 2014], o problema das p-medianas [Plastino et al., 2011], o problema do caixeiro viajante com coleta e entrega envolvendo um único tipo de produto [Guerine et al., 2016] e o problema de expansão de redes de transmissão de energia elétrica [González et al., 2018].

Posteriormente, em [Plastino et al., 2014], é proposta uma nova versão do DM-GRASP, chamada de Multi DM-GRASP (MDM-GRASP). A hipótese dos autores era de que minerar mais de uma vez poderia levar a extração de padrões mais refinados. O algoritmo funciona executando a mineração de dados toda vez que o conjunto elite se tornar estável, ou seja, quando não tiverem mudanças durante um determinado número de iterações. Além disso, quando o conjunto elite mudar e se tornar estável novamente, a mineração também é executada. Através de experimentos computacionais que compararam o GRASP, O DM-GRASP e o MDM-GRASP, foi possível perceber que o MDM-GRASP obteve os melhores resultados, não só em termos de qualidade das soluções, mas também em relação ao tempo computacional. A implementação do MDM-GRASP conseguiu achar melhores resultados para o problema de servidores para transmissão multicast confiável, em tempos computacionais menores [Plastino et al., 2014].

Além desta versão, em [Barbalho et al., 2011] [Barbalho et al., 2013] foram propostas duas novas variações, chamadas de DM-GRASP-PR e MDM-GRASP-PR, que combinam, de forma diferente, a mineração de dados com o GRASP, integrado com a técnica de reconexão por caminhos, existente na literatura. A técnica de reconexão por caminhos surgiu em [Glover et al., 2000] e consiste em uma estratégia de intensificação, onde caminhos são percorridos a partir de soluções elite para outras soluções elite. Durante o percurso, soluções intermediárias são geradas para que uma exploração eficiente do espaço de soluções seja realizada.

Estas abordagens foram aplicadas para o problema de síntese de redes com 2-caminhos. Os experimentos computacionais realizados pelos autores mostraram que o DM-GRASP-PR obteve melhores soluções em um tempo computacional menor que o GRASP com reconexão por caminhos original e o MDM-GRASP-PR obteve resultados ainda melhores que os encontrados pelo DM-GRASP-PR [Barbalho et al., 2011].

A Tabela 1 apresenta os trabalhos já publicados, que utilizam o DM-GRASP ou alguma variação:

Tabela 1: Trabalhos que utilizam o DM-GRASP e variações

Trabalho(s)	Problema	DM GRASP	Variação do DM-GRASP
[Ribeiro et al., 2004] [Ribeiro et al., 2006]	Empacotamento de Conjuntos	X	
[Santos et al., 2005]	Diversidade Máxima	X	
[Santos et al., 2006] [Plastino et al., 2014]	Replicação de Servidores para Transmissão Multicast Confiável	X	X
[Plastino et al., 2011]	P-Medianas	X	X
[Barbalho et al., 2011] [Barbalho et al., 2013]	Redes de 2 caminhos		X
[Guerine et al., 2016]	Caixeiro Viajante com Coleta e Entrega	X	
[González et al., 2018]	Expansão de Redes de Transmissão de Energia Elétrica	X	

Capítulo 3

Metodologia

Neste capítulo, que é dividido em 2 seções, os aspectos relacionados ao desenvolvimento deste trabalho serão explicados. Para todas as seções deste capítulo, considera-se como representação da solução, um vetor $\mathcal{M}$, que é composto por todos os itens $i \in I$ que forem alocados na mochila, resultando em uma solução viável para a instância p em questão. O tamanho do vetor irá variar de acordo com a instância.

Na Seção 3.1 a metaheurística GRASP e seus componentes, que serão implementados, são detalhados. Já na Seção 3.2, a ideia de como combinar o GRASP com a técnica de mineração de dados será apresentada, assim como o DM-GRASP implementado.

3.1 GRASP aplicado ao Problema da Mochila

A metaheurística GRASP [Feo et al., 1994] vem sendo utilizada com sucesso em diversos problemas clássicos de otimização combinatória, assim como aqueles ligados a situações do mundo real em diferentes áreas do conhecimento, como negócios, ciência e tecnologia [Resende and Ribeiro, 2003]. O algoritmo funciona de maneira iterativa, onde cada iteração é composta por duas fases: construção e busca local.

3.1.1 Heurística Construtiva

Na heurística construtiva, utiliza-se a estratégia de construir uma solução parcial, selecionando aleatoriamente um elemento por vez de uma Lista Restrita de Candidatos (LRC). Primeiramente, a LRC é construída através de uma estratégia gulosa. Para isso, é necessário definir um índice de performance, que nada mais é do que um critério de avaliação dos elementos que podem ser inseridos na solução. Neste trabalho, o índice de performance utilizado será a razão custo-benefício do item. Após a construção da LRC, a escolha aleatória dentre os melhores elementos daquela iteração é realizada, de forma que o índice de performance não seja o único critério determinante para decisão do elemento a ser inserido. A construção da solução

é realizada até que nenhum elemento possa ser adicionado na mochila, devido a sua restrição de capacidade. O índice de performance utilizado, representado por h_i , que é frequentemente utilizado na literatura, pode ser definido como:

$$h_i = \frac{v_i}{w_i}, \forall i \in I \quad (3.1)$$

Este índice mede o benefício associado à seleção de cada elemento. A heurística possui a característica de ser adaptativa pois o índice de performance adapta-se a solução já construída, isto é, a LRC é recalculada após cada iteração do GRASP. Uma vez definido o índice de performance, a LRC pode ser definida matematicamente como:

$$LRC = \{i \in I | h_{min} \leq h_i \leq h_{min} + \alpha(h_{max} - h_{min})\} \quad (3.2)$$

onde $h_{min} = \operatorname{argmin}_i \{h_i\}$, $h_{max} = \operatorname{argmax}_i \{h_i\}$ e α é um número real entre 0 e 1.

O valor de α influencia na qualidade e na diversidade da solução que será gerada no construtivo. Se o valor de α for zero, isto corresponde a um algoritmo puramente guloso, enquanto que $\alpha = 1$ corresponde a uma construção aleatória. Encontrar um valor para α apropriado é um fator de grande relevância para conseguir alcançar um bom balanceamento entre um bom tempo computacional e uma solução de qualidade. [Resende and Ribeiro, 2003]. Neste trabalho foram realizados diversos testes de valores para α , até chegar ao valor utilizado.

Utilizando as definições apresentadas acima, a fase de construção, aplicada ao problema da mochila, pode ser descrita através do Algoritmo 3.1.

Algoritmo 3.1: Construtivo(α)

```

1  início
2  | solução  $\mathcal{M} \leftarrow \emptyset$ ;
3  | capacidade  $C \leftarrow 0$ ;
4  | itensDisponiveis  $\leftarrow \emptyset$ ;
5  | capacidadeMochila  $\leftarrow$  capacidade da mochila da instância;
6  | enquanto  $C < \text{CapacidadeMochila} \wedge \text{ItensDisponiveis.size}() > 0$  faça
7  |   |  $h \leftarrow$  Gerar índices de performance;
8  |   |  $LRC \leftarrow$  Construir a Lista de Restrita Candidatos ( $h, \mathcal{M}, \alpha$ );
9  |   |  $\kappa \leftarrow$  Selecionar um item aleatório da LRC;
10 |   | se  $w_k + C \leq \text{capacidadeMochila}$  então
11 |   |   |  $\mathcal{M} \leftarrow \mathcal{M} \cup \{\kappa\}$ ;
12 |   |   |  $C \leftarrow C + w_k$ ;
13 |   |   | Remover item k do vetor itensDisponiveis;
14 |   | senão
15 |   |   | Remover item k do vetor itensDisponiveis;
16 |   | fim
17 | fim
18 | retornar  $\mathcal{M}$ ;
19 fim
```

A variável C representa a soma das capacidades dos itens que vão sendo alocados na mochila. O vetor `itensDisponiveis` é composto por todos os itens que estão disponíveis para serem alocados a mochila. Conforme as iterações forem rodando, este vetor é atualizado. Enquanto a capacidade dos itens que foram escolhidos para alocarem a mochila for menor que a capacidade máxima e ainda tiverem itens disponíveis, o algoritmo irá rodar.

3.1.2 Busca Local

A busca local inicia com uma solução viável da fase construtiva e tenta buscar por melhores soluções dentro de uma determinada vizinhança [Resende and Ribeiro, 2003]. O processo de explorar a vizinhança de uma solução do problema da mochila irá consistir na troca de itens que estão na mochila com itens que não estão na mochila.

A Figura 3 apresenta de forma genérica, uma possível troca de itens que ocorre na busca local. No exemplo, temos uma mochila que possui um vetor que representa 5 itens alocados a ela. Os itens que ficaram fora da mochila são representados por um vetor de candidatos. Ao constatar que ao trocar o primeiro item da mochila com o primeiro item do vetor de candidatos, a restrição de capacidade não é excedida e o valor da solução aumenta, os itens são trocados. Esse procedimento é feito realizando a combinação elemento a elemento, entre os dois vetores.

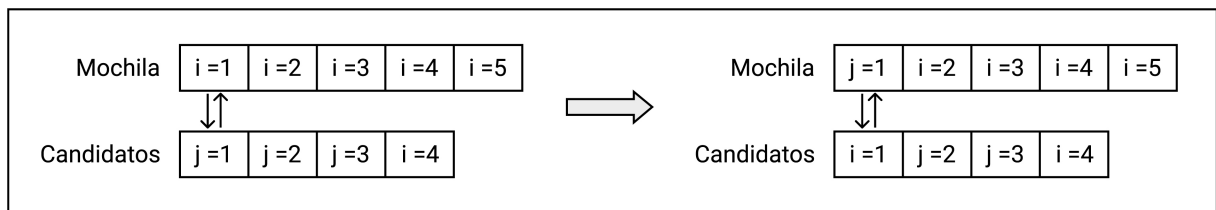


Figura 3: Exemplo de movimento da busca local

O Algoritmo 3.2 apresenta o pseudocódigo desta fase. Neste trabalho, a vizinhança é explorada totalmente, para que a melhor solução da vizinhança seja encontrada. Como parâmetro, é passada a solução $\mathcal{M}$ da fase de construção. Enquanto a solução $\mathcal{M}$ não for um ótimo local, as linhas 3 à 6 irão rodar. Primeiramente, na linha 3 é gerada uma solução vizinha de $\mathcal{M}$, que é definida pela variável t . Na linha 4 é testado se o custo (ou seja, o valor), da solução t é maior que o custo da solução base $\mathcal{M}$, ou seja, se $c(t) > c(\mathcal{M})$. Se for maior, é porque a solução t é melhor, portanto atualiza-se a solução $\mathcal{M}$, como pode ser visto na linha 5. Sempre que uma solução melhor é encontrada, a fase de busca local é reiniciada utilizando a solução recém-encontrada como solução base. A busca local irá terminar quando não há, na vizinhança, uma solução

melhor.

Algoritmo 3.2: Busca Local (Solução $\mathcal{M}$)

```

1 início
2   enquanto Solução  $\mathcal{M}$  não é um ótimo local faça
3     Encontrar Solução  $t$  em  $V(\mathcal{M})$ ;
4     se  $c(t) > c(\mathcal{M})$  então
5        $\mathcal{M} = t$ ;
6     fim
7   fim
8   retornar  $\mathcal{M}$ ;
9 fim

```

3.1.3 GRASP (Algoritmo completo)

De forma resumida, a primeira fase constrói uma solução viável inicial, enquanto que na segunda fase, a vizinhança da solução é explorada, com objetivo de melhorar a qualidade da solução inicial. Normalmente o critério de parada é estabelecido através de um número máximo de iterações ou de um limite de tempo. Neste trabalho, o critério de parada utilizado será o número de iterações, denominado *maxIter*.

O pseudocódigo do GRASP é apresentado através do Algoritmo 3.3. Primeiramente, é criada uma solução s , que é vazia no início. A melhor solução s^* será setada inicialmente como sendo a solução inicial s . Enquanto o número de iterações *maxIter* não for alcançado, as linhas 5 à 9 serão rodadas. A linha 5 representa a fase de construção, enquanto que a linha 6 corresponde a fase de busca local. Após as duas fases do GRASP, caso uma solução melhor seja encontrada, ou seja, se o custo $c(s)$ for maior que $c(s^*)$, a melhor solução s^* é atualizada. É importante ressaltar que a denominação custo está sendo utilizada para descrever a soma dos valores dos itens alocados na mochila e, quanto maior for este valor, melhor será a solução. Ao fim das iterações, a melhor solução é retornada.

3.2 Data Mining GRASP (DM-GRASP)

De forma geral, a incorporação da MD no GRASP ocorre em duas fases [González et al., 2018]. Na primeira fase, a heurística original é executada e soluções de alta qualidade são coletadas e armazenadas em um conjunto elite de soluções. Depois, a técnica de MD é aplicada no conjunto elite, mais precisamente no meio da execução da heurística, no intuito de extrair subconjuntos frequentes de itens. Os padrões minerados são aqueles que aparecem juntos com

Algoritmo 3.3: GRASP($\alpha, maxIter$)

```

1 início
2    $s \leftarrow \emptyset$ ;
3    $s^* \leftarrow s$ ;
4   enquanto maxIter não for alcançado faça
5      $s = \text{Construir a solução (parcial) gulosa aleatória}(\alpha)$ ;
6      $s = \text{Realizar a Busca Local}(s)$ ;
7     se  $c(s) > c(s^*)$  então
8        $s^* = s$ ;
9     fim
10  fim
11  retornar  $s^*$ ;
12 fim

```

uma frequência mínima, definida através da variável sup_{min} , que representa o suporte.

Na segunda fase, a ideia é utilizar os padrões minerados para guiar a busca por uma nova solução, de melhor qualidade. Isto ocorre através de uma modificação no algoritmo construtivo, onde os itens dos padrões frequentes são adicionados à mochila antes do início da construção. Desta forma, a mochila iniciará a fase de construção com os itens frequentes já adicionados a ela. Por fim, o pseudocódigo do DM-GRASP é apresentado no Algoritmo 3.4:

Algoritmo 3.4: DM-GRASP($\alpha, sup_{min}, maxIter$)

```

1 início
2   Instância de entrada;
3   enquanto  $1 \dots \frac{|maxIter|}{2}$  faça
4     Construir a solução gulosa aleatória;
5     Realizar a Busca Local;
6     Atualizar a melhor solução;
7     Atualizar o Conjunto Elite de Soluções;
8   fim
9    $listaPadroes \leftarrow \text{Minerar}(sup_{min}, \text{Conjunto Elite})$ ;
10  enquanto  $(\frac{|maxIter|}{2} + 1) \dots |maxIter|$  faça
11    Selecionar padrão  $p$  da listaPadroes;
12    Construir a solução de forma adaptada utilizando o padrão  $p$ ;
13    Realizar a Busca Local;
14    Atualizar a melhor solução;
15  fim
16  retornar a melhor solução;
17 fim

```

As linhas 3 à 8 representam a execução do GRASP original, onde o conjunto elite de soluções é criado. Na linha 9, os padrões são minerados através do algoritmo de mineração utilizado. Entre as linhas 10 e 15, o GRASP com a heurística construtiva adaptada é executado. O GRASP original é rodado durante metade das iterações ($\frac{|maxIter|}{2}$), enquanto que a outra metade

será rodada após a mineração de padrões frequentes. No final, a melhor solução é retornada.

É importante ressaltar que, neste trabalho, a base de dados é representada de forma que cada linha é composta por um id e um conjunto de itens. Os padrões frequentes a serem minerados correspondem a itens que aparecem juntos em soluções subótimas do problema da mochila, com determinada frequência. Cada transação t da base de dados D representa uma solução subótima do problema. Para minerar os padrões, será utilizado o algoritmo FPmax*. Este algoritmo utiliza um array para melhorar a performance de algoritmos que utilizam FP-trees, ou seja, representações da base de dados através da estrutura de árvore. Duas características de grande importância é de que ele é utilizado para testar subconjuntos, além de realizar diversas otimizações que contribuem para reduzir o tempo de execução [Grahne and Zhu, 2003].

A escolha do Problema da Mochila para validação da proposta deste trabalho se deve à compatibilidade do problema com a aplicação da MPF. Dois fatores principais contribuem com isso, o primeiro é que suas soluções se apresentam sob a forma de conjuntos de itens e o segundo pois a solução pode ser organizada como uma base de dados transacional.

Capítulo 4

Avaliação Experimental

Este capítulo apresenta os experimentos computacionais relacionados a comparação do DM-GRASP em relação ao GRASP, ou seja, da primeira metade das iterações com a segunda metade das iterações, onde a construção da solução já é iniciada com os padrões gerados. Para facilitar o processo de apresentação dos resultados, esta seção é dividida em três subseções. A Seção 4.1 apresenta uma comparação entre os valores da solução gerados pelas duas fases, a Seção 4.2 analisa o comportamento das duas fases graficamente e por último, a Seção 4.3 analisa a qualidade dos padrões frequentes que foram gerados durante o processo de Mineração de Dados.

4.1 GRASP vs DM-GRASP

Esta seção apresenta os experimentos computacionais que permitiram a avaliação da performance do DM-GRASP comparando-se com o GRASP. Os algoritmos foram implementados em C++11 e a técnica de mineração de padrões frequentes utilizada neste trabalho foi a FPmax* [Grahne and Zhu, 2003], com o código disponível em <http://fimi.ua.ac.be/src/>. Os testes foram executados em 333 instâncias que são benchmark na literatura.

Os tamanhos das instâncias variam de acordo com a quantidade de itens, que vão desde 10 itens até 1000 itens. Após diversos testes, os parâmetros definidos para serem utilizados foram fixados em: $\alpha = 0.15$, $maxIter = 250$ e $sup_{min} = 2$.

As Tabelas 2 e 3 apresentam os resultados obtidos. As tabelas reportam, para cada instância, a melhor solução obtida no GRASP e no DM-GRASP. Pode-se observar que as iterações do DM-GRASP resultaram em valores iguais ou superiores aos valores encontrados nas iterações da fase do GRASP sem a mineração de dados, em todas as instâncias utilizadas. De forma mais específica, o resultado foi igual em 44 instâncias e foi superior em 289 instâncias. Para melhor visualização, os valores superiores foram destacados em negrito nas tabelas. Portanto, é possível constatar que com a adição da mineração de dados, o GRASP tem uma melhora de performance significativa.

Tabela 2: Valores das soluções obtidos para as instâncias do Problema da Mochila

Instância	Estratégia		Instância	Estratégia		Instância	Estratégia	
	GRASP	DM-GRASP		GRASP	DM-GRASP		GRASP	DM-GRASP
6_100_1	136	136	120_100_10	2988	2989	320_100_10	3702	4121
6_100_2	166	166	140_100_1	578	705	340_100_1	8002	8107
6_100_3	156	156	140_100_2	3578	3578	340_100_2	4933	5219
6_100_4	148	148	140_100_3	3221	3270	340_100_3	5918	6331
6_100_5	188	188	140_100_4	2141	2352	340_100_4	6062	6374
6_100_6	116	116	140_100_5	3245	3272	340_100_5	6265	6505
6_100_7	152	152	140_100_6	754	934	340_100_6	1875	2300
6_100_8	128	128	140_100_7	1169	1318	340_100_7	5955	6225
6_100_9	108	108	140_100_8	780	990	340_100_8	7979	8058
6_100_10	118	118	140_100_9	1631	1794	340_100_9	2752	3179
10_100_1	306	306	140_100_10	2240	2339	340_100_10	8310	8456
10_100_2	287	287	160_100_1	4508	4512	360_100_1	4622	5055
10_100_3	223	223	160_100_2	2686	2823	360_100_2	5650	6064
10_100_4	269	269	160_100_3	897	1134	360_100_3	4606	5186
10_100_5	237	237	160_100_4	2410	2533	360_100_4	3976	4415
10_100_6	197	197	160_100_5	4022	4051	360_100_5	8581	8672
10_100_7	313	313	160_100_6	3303	3383	360_100_6	1324	1642
10_100_8	163	163	160_100_7	2617	2773	360_100_7	7641	7865
10_100_9	229	229	160_100_8	4040	4042	360_100_8	8570	8603
10_100_10	245	245	160_100_9	3513	3598	360_100_9	7980	8100
10_25_1	29	29	160_100_10	1130	1329	360_100_10	6852	7207
10_75_2	309	309	160_5_1	963	963	380_100_1	9455	9510
20_100_1	417	417	160_75_8	1552	1816	380_100_2	7739	7989
20_100_2	371	371	180_100_1	4325	4362	380_100_3	5429	5885
20_100_3	355	355	180_100_2	3729	3844	380_100_4	1242	1557
20_100_4	574	574	180_100_3	4319	4372	380_100_5	10178	10181
20_100_5	336	348	180_100_4	2374	2568	380_100_6	8209	8318
20_100_6	473	486	180_100_5	3231	3365	380_100_7	4028	4393
20_100_7	379	382	180_100_6	4054	4129	380_100_8	7149	7557
20_100_8	340	340	180_100_7	4625	4665	380_100_9	9456	9498
20_100_9	505	505	180_100_8	2197	2515	380_100_10	3358	3966
20_100_10	267	271	180_100_9	4154	4178	400_100_1	2734	3133
30_100_1	378	390	180_100_10	2933	3062	400_100_2	8898	9092
30_100_2	762	762	200_100_1	4201	4357	400_100_3	2112	2512
30_100_3	314	330	200_100_2	3025	3346	400_100_4	5491	5879
30_100_4	322	333	200_100_3	3236	3396	400_100_5	8108	8484
30_100_5	454	465	200_100_4	5442	5462	400_100_6	9658	9743
30_100_6	552	556	200_100_5	3915	4038	400_100_7	1031	1342
30_100_7	688	696	200_100_6	1041	1293	400_100_8	10298	10319
30_100_8	616	631	200_100_7	4843	4931	400_100_9	3747	4264
30_100_9	566	580	200_100_8	4129	4237	400_100_10	3582	4046
30_100_10	640	643	200_100_9	4677	4711	420_100_1	6238	6654
40_100_1	299	315	200_100_10	3768	3924	420_100_2	8276	8503
40_100_2	909	922	220_100_1	2862	3098	420_100_3	3990	4412
40_100_3	712	724	220_100_2	5038	5096	420_100_4	2428	2817
40_100_4	398	416	220_100_3	1418	1726	420_100_5	11220	11229
40_100_5	1236	1236	220_100_4	3901	4070	420_100_6	5760	6226
40_100_6	863	875	220_100_5	5240	5332	420_100_7	10261	10313
40_100_7	902	902	220_100_6	5245	5385	420_100_8	10736	10737
40_100_8	712	727	220_100_7	4713	4894	420_100_9	10900	10910
40_100_9	868	880	220_100_8	4926	5027	420_100_10	11160	11160
40_100_10	762	781	220_100_9	5273	5275	500_100_1	12004	12060
50_100_1	1232	1241	220_100_10	5372	5391	500_100_2	10654	10956
50_100_2	1254	1257	240_100_1	755	955	500_100_3	9475	9899
50_100_3	648	677	240_100_2	6319	6319	500_100_4	12757	12785
50_100_4	775	796	240_100_3	5244	5336	500_100_5	11734	11804
50_100_5	1204	1205	240_100_4	2730	3044	500_100_6	11400	11526
50_100_6	1209	1218	240_100_5	3445	3713	500_100_7	7354	7887
50_100_7	648	679	240_100_6	3680	4067	500_100_8	11653	11806

Tabela 3: Valores das soluções obtidos para as instâncias do Problema da Mochila

Instância	Estratégia		Instância	Estratégia		Instância	Estratégia	
	GRASP	DM-GRASP		GRASP	DM-GRASP		GRASP	DM-GRASP
50_100_8	702	734	240_100_7	6668	6668	500_100_9	11577	11672
50_100_9	1199	1200	240_100_8	6201	6201	500_100_10	8649	9118
50_100_10	1097	1128	240_100_9	5501	5550	600_100_1	8725	9222
60_100_1	1489	1518	240_100_10	783	927	600_100_2	15252	15278
60_100_2	1470	1476	260_100_1	3521	3825	600_100_3	7115	7876
60_100_3	1453	1481	260_100_2	3439	3766	600_100_4	6529	7111
60_100_4	1466	1469	260_100_3	5289	5466	600_100_5	14983	15021
60_100_5	1205	1237	260_100_4	515	775	600_100_6	13661	14027
60_100_6	1671	1674	260_100_5	4164	4398	600_100_7	2198	2711
60_100_7	1041	1081	260_100_6	2549	2878	600_100_8	7274	7854
60_100_8	1456	1460	260_100_7	5285	5498	600_100_9	3344	3948
60_100_9	681	774	260_100_8	1472	1770	600_100_10	9939	10589
60_100_10	316	366	260_100_9	5495	5695	700_100_1	13020	13575
70_100_1	1664	1681	260_100_10	6493	6538	700_100_2	4560	5436
70_100_2	1580	1607	260_25_1	1692	1692	700_100_3	11851	12493
70_100_3	1195	1215	280_100_1	2116	2508	700_100_4	8654	9516
70_100_4	993	1070	280_100_2	7135	7141	700_100_5	15843	16224
70_100_5	1188	1250	280_100_3	5380	5640	700_100_6	14548	14886
70_100_6	1526	1555	280_100_4	5671	5886	700_100_7	15920	16241
70_100_7	526	578	280_100_5	3052	3452	700_100_8	4566	5096
70_100_8	926	980	280_100_6	5099	5321	700_100_9	2925	3666
70_100_9	1778	1787	280_100_7	2820	3234	700_100_10	17043	17155
70_100_10	776	853	280_100_8	1830	2156	800_100_1	19761	19787
80_100_1	1852	1915	280_100_9	6746	6768	800_100_2	15677	16094
80_100_2	1125	1187	280_100_10	1000	1276	800_100_3	4711	5357
80_100_3	2074	2080	300_100_1	7438	7443	800_100_4	8060	8863
80_100_4	1944	1945	300_100_2	6389	6548	800_100_5	1169	1537
80_100_5	1340	1425	300_100_3	7011	7092	800_100_6	15982	16402
80_100_6	1881	1923	300_100_4	7583	7589	800_100_7	19364	19482
80_100_7	1935	1935	300_100_5	7404	7416	800_100_8	14849	15449
80_100_8	1924	1926	300_100_6	4002	4384	800_100_9	4077	4714
80_100_9	1414	1489	300_100_7	3395	3648	800_100_10	19556	19701
80_100_10	1467	1532	300_100_8	5357	5622	900_100_1	11328	12191
100_100_1	2238	2250	300_100_9	7698	7698	900_100_2	23322	23326
100_100_2	2130	2186	300_100_10	765	994	900_100_3	15043	15652
100_100_3	2352	2381	300_25_1	1646	1646	900_100_4	21132	21452
100_100_4	754	998	300_25_3	1584	1671	900_100_5	16764	17512
100_100_5	2327	2349	300_25_4	1622	1624	900_100_6	14878	15637
100_100_6	2216	2276	300_25_5	2079	2079	900_100_7	21658	21820
100_100_7	2048	2056	300_25_6	1920	1981	900_100_8	5457	6170
100_100_8	749	895	300_25_7	2181	2181	900_100_9	18497	18959
100_100_9	2290	2308	300_25_9	2004	2004	900_100_10	11032	11822
100_100_10	990	1113	300_75_10	5666	5730	1000_100_1	25493	25520
120_100_1	1430	1575	320_100_1	1193	1632	1000_100_2	2959	3598
120_100_2	766	899	320_100_2	5599	5944	1000_100_3	12171	13092
120_100_3	1746	1865	320_100_3	1209	1595	1000_100_4	13294	14071
120_100_4	2331	2380	320_100_4	3512	3937	1000_100_5	25252	25343
120_100_5	1504	1717	320_100_5	4421	4808	1000_100_6	22517	22986
120_100_6	1783	1892	320_100_6	5332	5735	1000_100_7	21659	22139
120_100_7	1743	1838	320_100_7	7905	7947	1000_100_8	23393	23641
120_100_8	418	509	320_100_8	1138	1378	1000_100_9	11976	12865
120_100_9	873	1035	320_100_9	7986	8049	1000_100_10	25515	25591

4.2 Análise do comportamento do DM-GRASP

Os gráficos da Figura 4, que foram gerados para 3 instâncias, permitem uma avaliação mais detalhada do comportamento do DM-GRASP. Neles, é ilustrada a variação do custo das soluções obtidas nas fases de construção e busca local, ao longo das 250 iterações. Os pontos roxos correspondem à fase de construção e os pontos azuis à fase de busca local.

Ao analisar os gráficos, eles nos permitem concluir que, após a extração dos padrões, que ocorre após a metade das iterações, a qualidade das soluções geradas na fase de construção aumenta de forma significativa. Na busca local, essa qualidade também aumenta. Isso reforça a ideia de que utilizar padrões faz com que a busca seja concentrada em regiões mais promissoras, aumentando a probabilidade de alcançar melhores soluções.

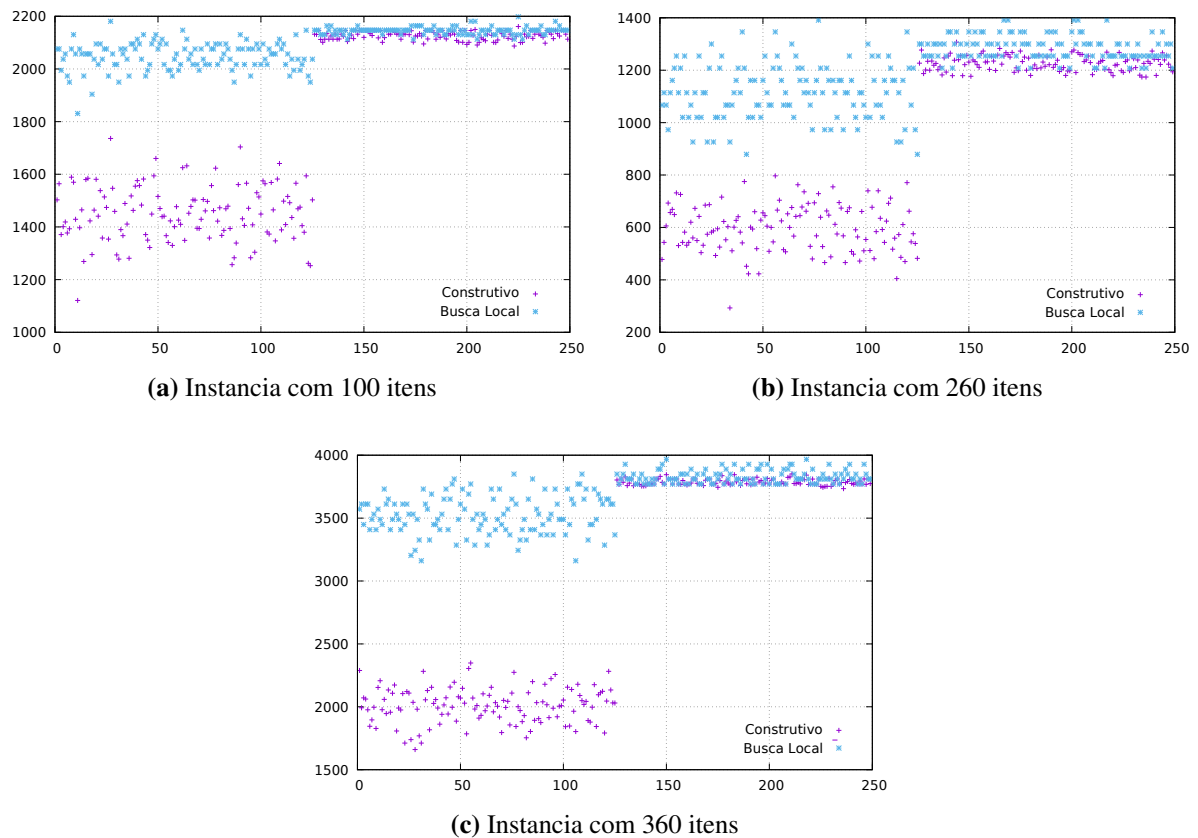


Figura 4: GRASP vs DM-GRASP

4.3 Análise da qualidade dos padrões frequentes

Para entender a qualidade dos padrões frequentes que foram gerados, é realizada uma comparação entre os itens que compõem os padrões de cada uma das instâncias, com os itens pertencentes a solução ótima da mesma. Para gerar as soluções ótimas, o pacote de software de otimização CPLEX¹ foi utilizado. O problema foi modelado como um modelo de programação linear inteira, conforme apresentado na Seção 2.3.

A comparação é realizada para cada um dos padrões, através do cálculo da porcentagem de itens que pertencem a solução ótima. Durante o processo de mineração de dados foram gerados até no máximo 5 padrões para cada instância, parâmetro que foi fixado no código. São apresentadas as piores e as melhores porcentagens de padrões para cada instância, além da média das porcentagens de todos os padrões.

Os resultados são apresentados nas Tabelas 4, 5, 6 e 7. É possível perceber que a grande maioria dos padrões possuem uma porcentagem bem alta, ou seja, a maior parte dos itens que compõem estes padrões pertencem a solução ótima, indicando que os padrões minerados são de alta qualidade. Ao sintetizar estes resultados com os resultados obtidos nas Seções 4.1 e 4.2, podemos perceber que os padrões frequentes contribuíram para valores de custo maiores.

¹<https://www.ibm.com/br-pt/products/ilog-cplex-optimization-studio>

Tabela 4: Porcentagens obtidas para cada uma das instâncias do Problema da Mochila

Instância		Padrões		
Quantidade de Itens	Id	Pior	Melhor	Média
6	6_100_1	100.00%	100.00%	100.00%
	6_100_2	80.00%	100.00%	92.00%
	6_100_3	60.00%	100.00%	87.00%
	6_100_4	75.00%	100.00%	91.67%
	6_100_5	80.00%	100.00%	88.00%
	6_100_6	66.70%	100.00%	86.67%
	6_100_7	100.00%	100.00%	100.00%
	6_100_8	80%	100.00%	95.00%
	6_100_9	80.00%	100.00%	95.00%
	6_100_10	75.00%	100.00%	93.75%
10	10_100_1	100.00%	100.00%	100.00%
	10_100_2	88.89%	100.00%	97.22%
	10_100_3	100.00%	100.00%	100.00%
	10_100_4	100.00%	100.00%	100.00%
	10_100_5	77.78%	100.00%	84.45%
	10_100_6	100.00%	100.00%	100.00%
	10_100_7	100.00%	100.00%	100.00%
	10_100_8	100.00%	100.00%	100.00%
	10_100_9	75.00%	87.50%	85.00%
	10_100_10	85.71%	100.00%	97.14%
20	10_75_2	75.0%	100.00%	95.0%
	10_25_1	57.14%	66.67%	62.86%
	20_100_1	94.74%	100.00%	98.95%
	20_100_2	92.86%	100.00%	97.14%
	20_100_3	83.33%	91.67%	87.18%
	20_100_4	100.00%	100.00%	100.00%
	20_100_5	88.89%	100.00%	97.78%
	20_100_6	92.31%	100.00%	98.46%
	20_100_7	85.71%	100.00%	92.64%
	20_100_8	86.67%	93.33%	91.81%
30	20_100_9	100.00%	100.00%	100.00%
	20_100_10	90.00%	100.00%	96.00%
	30_100_1	90.00%	100.00%	96.00%
	30_100_2	86.96%	91.30%	89.49%
	30_100_3	88.89%	100.00%	93.33%
	30_100_4	90.91%	100.00%	96.36%
	30_100_5	91.67%	100.00%	96.67%
	30_100_6	90.48%	100.00%	94.29%
	30_100_7	95.45%	100.00%	96.40%
	30_100_8	83.33%	88.89%	85.55%
40	30_100_9	93.75%	100.00%	96.40%
	30_100_10	95.65%	100.00%	96.63%
	40_100_1	87.50%	100.00%	92.50%
	40_100_2	96.67%	100.00%	98.69%
	40_100_3	87.50%	95.65%	92.31%
	40_100_4	83.33%	91.67%	89.55%
	40_100_5	97.37%	100.00%	98.95%
	40_100_6	96.55%	100.00%	97.24%
	40_100_7	90.91%	96.97%	95.13%
	40_100_8	95.83%	100.00%	97.53%
260	40_100_9	92.00%	96.15%	95.16%
	40_100_10	96.00%	100.00%	96.80%
	260_100_1	90.36%	94.05%	92.80%
	260_100_2	87.01%	90.91%	88.85%
	260_100_3	98.01%	100.00%	98.94%
	260_100_4	90.00%	100.00%	94.18%
	260_100_5	91.67%	94.85%	93.37%
	260_100_6	82.46%	87.93%	86.10%
	260_100_7	99.35%	100.00%	99.87%
	260_100_8	80%	84.38%	82.87%
280	260_100_9	99.34%	100.00%	99.60%
	260_100_10	99.53%	100.00%	99.62%
	260_25_1	81.71%	83.75%	82.12%
	280_100_1	79.55%	84.09%	82.73%
	280_100_2	98.60%	100.00%	99.55%
	280_100_3	99.55%	98.62%	98.61%
	280_100_4	98.68%	99.34%	99.21%
	280_100_5	89.86%	92.86%	91.67%
	280_100_6	96.90%	99.22%	98.28%
	280_100_7	85.71%	90.77%	87.72%
300	280_100_8	81.58%	84.21%	82.73%
	280_100_9	99.21%	99.60%	99.52%
	280_100_10	68.42%	73.68%	70.10%
	300_25_1	61.17%	61.17%	61.17%
	300_25_3	97.96%	100.0%	98.38%
	300_25_4	78.05%	79.01%	78.63%
	300_25_5	58.45%	60.58%	59.55%
	300_25_6	95.52%	98.48%	97.01%
	300_25_7	68.85%	71.19%	70.13%
	300_25_9	70.27%	73.58%	72.11%
320	300_75_10	97.64%	98.58%	98.02%
	300_100_1	99.65%	100.00%	99.93%
	300_100_2	98.95%	100.00%	99.37%
	300_100_3	99.13%	100.00%	99.65%
	300_100_4	99.65%	100.00%	99.93%
	300_100_5	99.26%	100.0%	99.63%
	300_100_6	94.57%	96.74%	95.69%
	300_100_7	85.53%	86.84%	86.28%
	300_100_8	95.56%	97.01%	96.43%
	300_100_9	99.66%	100.0%	99.8%
300	300_100_10	71.43%	80.0%	76.71%
	320_100_1	63.64%	72.73%	68.46%
	320_100_2	97.08%	98.53%	97.95%
	320_100_3	70.83%	75.0%	72.73%
	320_100_4	87.34%	89.87%	88.22%
	320_100_5	85.15%	87.13%	86.0%
	320_100_6	97.83%	98.56%	97.99%
	320_100_7	99.64%	100.0%	99.93%
	320_100_8	71.43%	76.19%	72.64%
	320_100_9	99.61%	100.0%	99.92%
300	320_100_10	85.37%	90.12%	87.72%
	340_100_1	99.2%	100.0%	99.77%
	340_100_2	88.7%	91.45%	90.24%
300	340_100_3	98.06%	98.73%	98.59%

Tabela 5: Porcentagens obtidas para cada uma das instâncias do Problema da Mochila

Instância		Padrões		
Quantidade de Itens	Id	Pior	Melhor	Média
50	50_100_1	97.67%	100.0%	98.14%
	50_100_2	95.45%	100.0%	97.73%
	50_100_3	84.21%	94.74%	90.47%
	50_100_4	86.36%	95.45%	92.64%
	50_100_5	95.74%	100.0%	97.86%
	50_100_6	91.67%	97.14%	94.86%
	50_100_7	94.44%	100.0%	97.84%
	50_100_8	100.0%	100.0%	100.0%
	50_100_9	95.56%	100.0%	97.75%
	50_100_10	90.91%	96.97%	95.72%
60	60_100_1	95.35%	100.0%	97.66%
	60_100_2	98.11%	100.0%	99.24%
	60_100_3	97.87%	100.0%	99.57%
	60_100_4	98.18%	100.0%	98.91%
	60_100_5	99.63%	99.64%	99.64%
	60_100_6	98.15%	100.0%	98.54%
	60_100_7	86.67%	100.0%	93.24%
	60_100_8	97.96%	100.0%	98.78%
	60_100_9	80.0%	86.67%	83.17%
	60_100_10	100.0%	100.0%	100.0%
70	70_100_1	96.36%	96.43%	96.4%
	70_100_2	97.87%	100.0%	99.57%
	70_100_3	94.44%	100.0%	98.32%
	70_100_4	96.0%	100.0%	99.2%
	70_100_5	100.0%	100.0%	100.0%
	70_100_6	98.04%	100.0%	99.22%
	70_100_7	91.67%	100.0%	98.33%
	70_100_8	95.65%	100.0%	98.3%
	70_100_9	98.33%	100.0%	98.67%
	70_100_10	90.0%	95.0%	93.66%
80	80_100_1	98.08%	100.0%	98.87%
	80_100_2	93.1%	93.33%	93.19%
	80_100_3	94.59%	98.63%	96.73%
	80_100_4	97.4%	100.0%	98.43%
	80_100_5	94.29%	100.0%	98.29%
	80_100_6	100.0%	100.0%	100.0%
	80_100_7	96.1%	98.7%	97.4%
	80_100_8	97.3%	98.65%	98.11%
	80_100_9	94.59%	100.0%	98.92%
	80_100_10	94.74%	100.0%	98.42%
100	100_100_1	98.8%	100.0%	99.52%
	100_100_2	98.41%	100.0%	99.05%
	100_100_3	97.26%	100.0%	98.35%
	100_100_4	94.44%	100.0%	96.66%
	100_100_5	98.75%	100.0%	99.26%
	100_100_6	100.0%	100.0%	100.0%
	100_100_7	95.4%	98.82%	96.99%
	100_100_8	81.25%	94.12%	90.07%
	100_100_9	96.0%	97.37%	97.07%
	100_100_10	86.96%	91.3%	90.35%
340	340_100_4	97.42%	98.73%	97.82%
	340_100_5	97.53%	98.15%	97.79%
340	340_100_6	72.97%	78.38%	75.67%
	340_100_7	96.64%	98.01%	97.46%
	340_100_8	99.61%	100.0%	99.84%
	340_100_9	78.57%	81.82%	80.15%
	340_100_10	99.61%	100.0%	99.77%
360	360_100_1	89.32%	91.35%	90.29%
	360_100_2	92.75%	96.38%	94.93%
	360_100_3	91.51%	93.33%	92.3%
	360_100_4	84.88%	89.53%	86.71%
	360_100_5	99.63%	99.64%	99.64%
	360_100_6	70.37%	76.92%	74.47%
	360_100_7	98.13%	99.53%	98.51%
	360_100_8	99.03%	99.35%	99.29%
	360_100_9	98.29%	99.57%	98.98%
	360_100_10	95.88%	98.82%	97.17%
380	380_100_1	99.68%	100.0%	99.94%
	380_100_2	98.17%	99.09%	98.72%
	380_100_3	92.42%	93.89%	93.44%
	380_100_4	65.22%	70.83%	67.79%
	380_100_5	99.73%	100.0%	99.95%
	380_100_6	98.94%	99.3%	99.16%
	380_100_7	81.82%	84.09%	83.3%
	380_100_8	96.13%	96.7%	96.25%
	380_100_9	99.07%	100.0%	99.75%
	380_100_10	85.71%	90.0%	87.71%
400	400_100_1	75.86%	77.97%	77.32%
	400_100_2	98.88%	99.25%	98.95%
	400_100_3	69.05%	73.81%	71.43%
	400_100_4	91.6%	93.13%	92.5%
	400_100_5	98.21%	99.11%	98.83%
	400_100_6	99.36%	99.68%	99.49%
	400_100_7	78.95%	80.0%	79.16%
	400_100_8	99.18%	99.73%	99.56%
	400_100_9	77.5%	79.01%	78.3%
	400_100_10	80.77%	82.05%	81.33%
420	420_100_1	90.48%	91.78%	91.37%
	420_100_2	96.43%	98.67%	97.69%
	420_100_3	82.95%	83.52%	83.06%
	420_100_4	77.55%	81.63%	80.01%
	420_100_5	99.24%	99.74%	99.49%
	420_100_6	89.63%	93.28%	91.85%
	420_100_7	99.71%	100.0%	99.83%
	420_100_8	99.52%	99.76%	99.62%
	420_100_9	99.74%	100.0%	99.79%
	420_100_10	99.76%	100.0%	99.81%
500	500_100_1	100.0%	100.0%	100.0%
	500_100_2	98.98%	100.0%	99.25%

Tabela 6: Porcentagens obtidas para cada uma das instâncias do Problema da Mochila

Instância		Padrões		
Quantidade de Itens	Id	Pior	Melhor	Média
120	120_100_1	93.94%	97.06%	95.28%
	120_100_2	83.33%	94.12%	88.43%
	120_100_3	90.48%	95.24%	93.34%
	120_100_4	97.18%	98.59%	98.03%
	120_100_5	97.22%	100.0%	98.33%
	120_100_6	88.37%	88.64%	88.48%
	120_100_7	95.24%	97.73%	96.21%
	120_100_8	88.89%	100.0%	93.96%
	120_100_9	84.21%	89.47%	88.07%
	120_100_10	99.12%	100.0%	99.48%
140	140_100_1	76.92%	91.67%	83.72%
	140_100_2	99.26%	100.0%	99.85%
	140_100_3	98.02%	99.01%	98.8%
	140_100_4	90.0%	96.08%	94.75%
	140_100_5	98.23%	100.0%	99.65%
	140_100_6	93.33%	100.0%	96.0%
	140_100_7	81.48%	89.66%	86.98%
	140_100_8	82.35%	88.24%	84.71%
	140_100_9	86.84%	92.11%	89.47%
	140_100_10	98.21%	100.0%	99.64%
160	160_100_1	98.68%	99.34%	98.94%
	160_100_2	93.75%	98.46%	95.71%
	160_100_3	82.35%	83.33%	83.13%
	160_100_4	96.83%	100.0%	98.74%
	160_100_5	99.23%	100.0%	99.85%
	160_100_6	98.91%	98.92%	98.91%
	160_100_7	97.26%	98.65%	98.08%
	160_100_8	96.64%	97.97%	97.31%
	160_100_9	97.17%	100.0%	99.24%
	160_100_10	79.17%	91.67%	85.83%
180	160_75_8	88.57%	94.29%	90.46%
	160_5_1	25.97%	26.32%	26.25%
	180_100_1	99.29%	100.0%	99.72%
	180_100_2	96.26%	99.06%	97.56%
	180_100_3	98.55%	100.0%	99.71%
	180_100_4	94.64%	98.21%	96.1%
	180_100_5	96.67%	98.89%	98.01%
	180_100_6	99.17%	100.0%	99.67%
	180_100_7	98.62%	100.0%	99.17%
	180_100_8	89.8%	96.0%	92.61%
200	180_100_9	97.2%	98.61%	97.65%
	180_100_10	97.3%	98.68%	98.39%
	200_100_1	98.39%	99.19%	99.03%
	200_100_2	92.75%	97.26%	94.56%
	200_100_3	100.0%	100.0%	100.0%
	200_100_4	99.44%	99.44%	99.44%
	200_100_5	97.37%	99.13%	98.6%
	200_100_6	86.36%	90.91%	89.12%
	200_100_7	98.52%	100.0%	99.56%
	200_100_8	97.69%	99.24%	98.92%
500	200_100_9	99.41%	100.0%	99.88%
	200_100_10	98.1%	100.0%	98.68%
	500_100_3	96.75%	97.54%	97.22%
	500_100_4	99.56%	100.0%	99.69%
	500_100_5	99.28%	99.52%	99.33%
	500_100_6	98.6%	100.0%	99.61%
	500_100_7	88.51%	90.12%	89.25%
	500_100_8	99.43%	100.0%	99.71%
	500_100_9	99.5%	100.0%	99.7%
	500_100_10	94.62%	97.76%	96.15%
600	600_100_1	89.66%	90.78%	90.38%
	600_100_2	99.82%	100.0%	99.93%
	600_100_3	85.53%	86.79%	86.18%
	600_100_4	81.75%	85.51%	84.2%
	600_100_5	100.0%	100.0%	100.0%
	600_100_6	99.24%	100.0%	99.59%
	600_100_7	61.9%	69.77%	65.7%
	600_100_8	84.05%	85.8%	84.66%
	600_100_9	73.44%	79.69%	76.32%
	600_100_10	95.53%	95.98%	95.87%
700	700_100_1	97.93%	99.11%	98.51%
	700_100_2	76.09%	80.43%	77.71%
	700_100_3	92.91%	95.07%	94.2%
	700_100_4	85.05%	86.22%	85.42%
	700_100_5	98.48%	98.92%	98.74%
	700_100_6	98.77%	99.5%	99.06%
	700_100_7	99.14%	99.57%	99.44%
	700_100_8	73.33%	76.34%	75.05%
	700_100_9	68.97%	72.41%	70.94%
	700_100_10	99.27%	100.0%	99.63%
800	800_100_1	99.45%	99.59%	99.53%
	800_100_2	97.69%	98.15%	97.92%
	800_100_3	68.13%	73.4%	70.7%
	800_100_4	79.78%	81.36%	80.57%
	800_100_5	47.62%	61.9%	54.2%
	800_100_6	97.7%	98.39%	98.02%
	800_100_7	99.54%	99.85%	99.66%
	800_100_8	96.12%	96.88%	96.57%
	800_100_9	66.67%	74.07%	70.05%
	800_100_10	99.37%	99.84%	99.69%
900	900_100_1	87.26%	87.98%	87.67%
	900_100_2	99.77%	99.89%	99.84%
	900_100_3	94.74%	96.04%	95.31%
	900_100_4	98.33%	98.99%	98.73%
	900_100_5	95.72%	96.84%	96.35%
	900_100_6	92.92%	93.66%	93.23%
	900_100_7	99.57%	99.86%	99.71%
	900_100_8	69.09%	72.07%	70.3%
	900_100_9	96.99%	98.2%	97.88%
	900_100_10	84.86%	86.0%	85.32%
1000	1000_100_1	100.0%	100.0%	100.0%
	1000_100_2	64.29%	66.67%	65.83%
	1000_100_3	86.59%	87.73%	87.17%

Tabela 7: Porcentagens obtidas para cada uma das instâncias do Problema da Mochila

Instância		Padrões		
Quantidade de Itens	Id	Pior	Melhor	Média
220	220_100_1	89.55%	92.42%	91.21%
	220_100_2	100.0%	100.0%	100.0%
	220_100_3	82.76%	93.33%	87.82%
	220_100_4	98.04%	99.03%	98.63%
	220_100_5	99.38%	100.0%	99.88%
	220_100_6	99.31%	100.0%	99.73%
	220_100_7	95.93%	96.77%	96.59%
	220_100_8	97.97%	99.32%	98.38%
	220_100_9	99.53%	100.0%	99.72%
	220_100_10	97.95%	98.46%	98.36%
240	240_100_1	82.35%	93.33%	88.47%
	240_100_2	100.00%	100.00%	100.00%
	240_100_3	99.38%	100.00%	99.63%
	240_100_4	86.89%	89.06%	88.45%
	240_100_5	90.12%	92.68%	91.91%
	240_100_6	94.51%	96.70%	94.98%
	240_100_7	100.00%	100.00%	100.00%
	240_100_8	99.58%	99.58%	99.58%
	240_100_9	98.37%	99.46%	98.81%
	240_100_10	66.67%	75.00%	72.33%

Instância		Padrões		
Quantidade de Itens	Id	Pior	Melhor	Média
1000	1000_100_4	86.56%	88.24%	87.71%
	1000_100_5	99.64%	99.76%	99.71%
	1000_100_6	98.78%	99.85%	99.42%
	1000_100_7	98.26%	98.72%	98.53%
	1000_100_8	99.33%	99.6%	99.49%
	1000_100_9	86.04%	87.04%	86.64%
	1000_100_10	99.78%	100.0%	99.89%

Capítulo 5

Conclusões e Trabalhos Futuros

O objetivo deste trabalho foi de estudar o impacto da inserção de uma técnica de mineração de dados na metaheurística GRASP, que é muito conhecida na literatura. Para isso, o DM-GRASP, método híbrido já existente e que realiza esta combinação, foi implementado. O código foi aplicado a um problema de otimização combinatória muito conhecido e importante, o Problema da Mochila.

Ao analisar os trabalhos relacionados, é possível perceber que o DM-GRASP já foi explorado para diversos problemas, além de possuir algumas variações. Contudo, nestes trabalhos, entender qual porcentagem dos itens que foram escolhidos para compor os padrões realmente fazem parte da solução ótima, é algo que não é analisado. No presente trabalho, além da performance das duas fases do DM-GRASP ser comparada, através das Seções 4.1 e 4.2, os padrões frequentes também são analisados na Seção 4.3. Nota-se que as porcentagens obtidas foram altas, o que indicou que os padrões são de qualidade, pois são bem próximos da solução ótima, ou até iguais.

Com os experimentos computacionais, realizados em 333 instâncias, foi possível perceber que os resultados contribuíram para a ideia de que combinar meta-heurísticas com técnicas de mineração de dados não é somente efetivo, mas também desejado para a busca da otimalidade.

Como trabalhos futuros, é possível destacar:

- Estender os testes para outras instâncias da literatura
- Estender os testes para variações do Problema da Mochila, como por exemplo o Problema da Mochila Quadrática
- Realizar comparações de tempos computacionais para instâncias mais complexas
- Incorporar a técnica de mineração de dados em outras metaheurísticas conhecidas

Referências Bibliográficas

- E. Aarts, E. H. Aarts, and J. K. Lenstra. *Local search in combinatorial optimization*. Princeton University Press, 2003. 1, 4, 6
- M. Assi and R. A. Haraty. A Survey of the Knapsack Problem. *ACIT 2018 - 19th International Arab Conference on Information Technology*, pages 1–6, 2019. doi: 10.1109/ACIT.2018.8672677. 1
- H. Barbalho, I. Rosseti, S. Martins, and A. Plastino. Um grasp híbrido com reconexão por caminhos e mineração de dados. *Revista Eletrônica de Iniciação Científica em Computação*, 11(3), 2011. 11, 12
- H. Barbalho, I. Rosseti, S. L. Martins, and A. Plastino. A hybrid data mining grasp with path-relinking. *Computers Operations Research*, 40(12):3159 – 3173, 2013. ISSN 0305-0548. doi: <https://doi.org/10.1016/j.cor.2012.02.022>. URL <http://www.sciencedirect.com/science/article/pii/S0305054812000482>. 2, 11, 12
- R. Bellman. Dynamic programming princeton university press princeton. *New Jersey Google Scholar*, 1957. 7
- C. Blum and A. Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, 35(3):268–308, 2003. 6
- C. Blum and A. Roli. Hybrid metaheuristics: an introduction. In *Hybrid metaheuristics*, pages 1–30. Springer, 2008. 2, 6, 8
- E. Chong and S. Zak. *An Introduction to Optimization*. Wiley Series in Discrete Mathematics and Optimization. Wiley, 2013. ISBN 9781118515150. URL <https://books.google.com.br/books?id=iD5s0iKXHP8C>. 5
- G. B. Dantzig. Discrete-variable extremum problems. *Operations research*, 5(2):266–288, 1957. 1
- T. Feo and M. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6:109–133, 03 1995. doi: 10.1007/BF01096763. 1

- T. A. Feo, M. G. Resende, and S. H. Smith. A greedy randomized adaptive search procedure for maximum independent set. *Operations Research*, 42(5):860–878, 1994. v, vi, 2, 8, 13
- F. Glover. Future paths for integer programming and links to artificial intelligence. *Computers operations research*, 13(5):533–549, 1986. 8
- F. Glover, M. Laguna, and R. Martí. Fundamentals of scatter search and path relinking. *Control and cybernetics*, 29(3):653–684, 2000. 11
- F. W. Glover and G. A. Kochenberger. *Handbook of metaheuristics*, volume 57. Springer Science & Business Media, 2006. v, vi
- B. Goethals. Advances in frequent itemset mining implementations: Introduction to fimi03. In *Proc. IEEE ICDM Workshop on FIMI'03*, 2003. 10
- M. Goldbarg. *Otimização Combinatória e Programação Linear*. 01 2005. ISBN 85-352-1520-4. viii, 4, 5
- M. Goldbarg and H. Luna. *Otimização combinatória e programação linear: modelos e algoritmos*. CAMPUS - RJ, 2005. ISBN 9788535215205. URL <https://books.google.com.br/books?id=Q-bNGAAACAAJ>. v, 1, 5, 7, 8
- D. E. Goldenberg. Genetic algorithms in search, optimization and machine learning, 1989. 8
- P. H. González, A. F. U. Macambira, R. V. Pinto, L. Simonetti, N. Maculan, and P. Michelon. New proposals for modelling and solving the problem of covering solids using spheres of different radii. *RAIRO - Operations Research*, 2019. 2
- P. H. González, M. Guerine, and J. S. Brandão. Avaliação do impacto da inserção de mineração de dados na resolução do problema de expansão de redes de transmissão de energia elétrica. In *Anais do XLVIII SBPO - Simpósio Brasileiro de Pesquisa Operacional*. SOBRAPO, 2018. 2, 11, 12, 16
- G. Grahne and J. Zhu. Efficiently using prefix-trees in mining frequent itemsets. In *Frequent Itemset Mining Implementations (FIMI'03)*. Proceedings of the ICDM 2003 Workshop on Frequent Itemset Mining Implementations, 2003. 18, 19
- M. Guerine, I. Rosseti, and A. Plastino. Extending the hybridization of metaheuristics with data mining: Dealing with sequences. *Intelligent Data Analysis*, 20(5):1133–1156, 2016. 11, 12

- J. Han, J. Pei, and M. Kamber. *Data mining: concepts and techniques*. Elsevier, 2011. 2, 9
- D. J. Hand and N. M. Adams. Data mining. *Wiley StatsRef: Statistics Reference Online*, pages 1–7, 2014. 9
- F. Hillier and G. Lieberman. *Introdução à Pesquisa Operacional*. AMGH, 2013. ISBN 9788580551198. URL <https://books.google.com.br/books?id=-A88a0-KxQ0C>. 5
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983. 8
- A. H. Land and A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960. ISSN 00129682, 14680262. URL <http://www.jstor.org/stable/1910129>. 7
- Y.-H. Liu. Mining frequent patterns from univariate uncertain data. *Data Knowledge Engineering*, 71(1):47 – 68, 2012. ISSN 0169-023X. doi: <https://doi.org/10.1016/j.datak.2011.07.009>. URL <http://www.sciencedirect.com/science/article/pii/S0169023X11001066>. 9
- M. Maia, A. Plastino, and P. Penna. Incorporando mineração de dados a uma heurística multi-start ils para o problema de roteamento de veículos com frota heterogênea. 08 2015. 10
- G. R. Mauri, G. Climaco, B. S. Vieira, G. M. Ribeiro, P. H. González, N. F. R. C. Júnior, L. A. d. R. T. Júnior, and A. d. O. Nunes. Meta-heurística cs para o problema de localização de contadores de tráfego em redes de transporte. In *Anais do XLVIII SBPO - Simpósio Brasileiro de Pesquisa Operacional*. SOBRAPO, 2018. 2
- N. Mladenović and P. Hansen. Variable neighborhood search. *Computers & operations research*, 24(11):1097–1100, 1997. 8
- I. H. Osman and G. Laporte. *Metaheuristics: A bibliography*, 1996. 8
- C. H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, Inc., USA, 1982. ISBN 0131524623. 5
- A. Plastino, R. Fuchshuber, S. d. L. Martins, A. A. Freitas, and S. Salhi. A hybrid data mining metaheuristic for the p-median problem. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 4(3):313–335, 2011. 11, 12

- A. Plastino, H. Barbalho, L. Santos, R. Fuchshuber, and S. Martins. Adaptive and multi-mining versions of the dm-grasp hybrid metaheuristic. *Journal of Heuristics*, 20:39–74, 02 2014. doi: 10.1007/s10732-013-9231-0. 2, 11, 12
- J. Puchinger and G. R. Raidl. Combining metaheuristics and exact algorithms in combinatorial optimization: A survey and classification. In *International work-conference on the interplay between natural and artificial computation*, pages 41–53. Springer, 2005. 7
- G. R. Raidl. A unified view on hybrid metaheuristics. In *International workshop on hybrid metaheuristics*, pages 1–12. Springer, 2006. 8
- M. G. Resende and C. C. Ribeiro. Greedy randomized adaptive search procedures. In *Handbook of metaheuristics*, pages 219–249. Springer, 2003. 13, 14, 15
- M. Ribeiro, V. Trindade, A. Plastino, and S. Martins. Hybridization of grasp metaheuristics with data mining techniques. pages 69–78, 01 2004. v, vi, 10, 12
- M. H. Ribeiro, A. Plastino, and S. L. Martins. Hybridization of grasp metaheuristic with data mining techniques. *Journal of Mathematical Modelling and Algorithms*, 5(1):23–41, 2006. v, vi, 10, 12
- L. F. Santos, M. H. Ribeiro, A. Plastino, and S. L. Martins. A hybrid grasp with data mining for the maximum diversity problem. In M. J. Blesa, C. Blum, A. Roli, and M. Sampels, editors, *Hybrid Metaheuristics*, pages 116–127, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg. ISBN 978-3-540-31898-9. 9, 10, 12
- L. F. Santos, R. Milagres, C. V. Albuquerque, S. Martins, and A. Plastino. Ngl01-4: A hybrid grasp with data mining for efficient server replication for reliable multicast. In *IEEE Globecom 2006*, pages 1–6, 2006. doi: 10.1109/GLOCOM.2006.246. 11, 12
- L. F. Santos, S. L. Martins, and A. Plastino. Applications of the dm-grasp heuristic: a survey. *International Transactions in Operational Research*, 15(4):387–416, 2008. 10
- P. H. G. Silva, A. F. U. Macambira, R. V. Pinto, L. Simonetti, N. Maculan, and P. Michelon. New proposals for modelling and solving the problem of covering solids using spheres of different radii. *RAIRO-Operations Research*, 54(3):873–882, 2020. 9, 10
- E. D. Taillard. Ant systems. *Handbook of Applied Optimization*, pages 130–137, 2002. 8

- E.-G. Talbi. A taxonomy of hybrid metaheuristics. *Journal of Heuristics*, 8:541–564, 01 2002. doi: 10.1023/A:1016540724870. 2
- T. Weise. Global optimization algorithms-theory and application (2009). *Kostenloses elektronisches Buch über Optimierungsverfahren, Evolutionäre Algorithmen und andere Metaheuristiken*. Online verfügbar unter <http://www.it-weise.de/>(gelesen 2008-12-13), 2007. viii, 6, 7
- I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal. *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann, Amsterdam, 4 edition, 2017. ISBN 978-0-12-804291-5. URL <http://www.sciencedirect.com/science/book/9780128042915>. 2, 9